AUTH_STORE.PY LINE-BY-LINE EXPLANATION

auth_store.py is a small JSON-backed auth helper. It loads and saves users from instance/users.json, normalizes phone numbers, exposes safe public user data, and creates or updates phone/Google users.

IMPORTS AND PATHS

Lines 1-5: import helpers for JSON storage, regex cleanup, UUID generation, UTC timestamps, and filesystem paths.

Line 8: sets DATA_DIR to an instance folder next to auth_store.py.

Line 9: sets USERS_PATH to instance/users.json.

TIME HELPER

Lines 12-13: _now_iso() returns the current UTC time as an ISO string, with microseconds removed.

LOADING USERS

Lines 16-18: _load_users() returns an empty list if users.json does not exist.

Lines 20-23: tries to read and parse the JSON file; if reading or parsing fails, returns an empty list.

Lines 25-28: extracts the users list from the JSON payload, verifies it is actually a list, then returns it.

SAVING USERS

Lines 31-32: _save_users(users) ensures the instance directory exists.

Lines 33-36: writes {"users": users} to users.json as formatted UTF-8 JSON.

PHONE NORMALIZATION

Lines 39-42: normalize_phone(phone) strips the input; empty input returns an empty string.

Line 44: removes spaces, parentheses, dots, and hyphens.

Lines 45-46: converts international 00... prefix into +....

Lines 48-50: if the number starts with +, keeps only digits after it and returns +digits

only if there are at least 8 digits.

Lines 52-54: for a 10-digit local number, assumes India and prefixes +91.

Lines 55-57: for other numbers with at least 8 digits, prefixes +; otherwise returns an empty string.

INITIALS HELPER

Lines 60-61: _initials(name, fallback) splits the name into non-empty words.

Lines 62-63: for two or more name parts, returns first initial plus last initial.

Lines 64-65: for one name part, returns its first two characters.

Lines 67-68: if no name exists, uses the fallback value or U.

PUBLIC USER SHAPE

Lines 71-73: public_user(user) returns None if no user is provided.

Line 75: chooses a display name from name, email, phone, or User.

Line 76: chooses auth method from auth_method, provider, or defaults to phone.

Line 77: builds a readable label: Google account or Phone OTP.

Lines 79-91: returns a sanitized user dictionary for frontend/session use, including id, name, contact fields, avatar, auth info, initials, verified status, and timestamps.

FIND HELPERS

Lines 94-95: find_user_by_id(user_id) loads users and returns the first matching id.

Lines 98-103: find_user_by_phone(phone) normalizes the phone first, then finds a matching stored phone.

Lines 106-111: find_user_by_email(email) lowercases/strips email, then finds a matching stored email.

Lines 114-118: find_user_by_google_sub(google_sub) finds a user by Google subject id.

PHONE LOGIN/CREATE

Lines 121-124: upsert_phone_user(name, phone) normalizes the phone, loads users, and gets the current timestamp.

Lines 126-134: if a user with that phone exists, it updates name if provided, marks auth as phone, verifies the user, updates last_login_at, saves, and returns the user.

Lines 136-146: if no user exists, creates a new phone user with a UUID, name or phone fallback, empty email/avatar, verified status, and timestamps.

Lines 147-149: appends the new user, saves all users, and returns it.

GOOGLE LOGIN/CREATE

Lines 152-155: upsert_google_user(...) normalizes email, loads users, and gets the current timestamp.

Lines 157-162: first tries to find an existing user by Google subject id.

Lines 163-167: if none was found, tries to find an existing user by email.

Lines 169-178: if an existing user is found, updates Google id, email, name, avatar, auth method, verified status, and login time, then saves and returns it.

Lines 180-191: if no user exists, creates a new Google user with UUID, Google id, name/email fallback, empty phone, avatar, verified status, and timestamps.

Lines 192-194: appends the new user, saves all users, and returns it.

IMPORTANT DETAIL

This file uses a simple JSON file as the user store. That is fine for a small/demo app, but it has no locking, indexing, or database-level protection for concurrent writes.